

Universidad de los Llanos  
Facultad de Ciencias Básicas e Ingeniería  
Ingeniería de Software

# Documento Técnico

Auditoría y Rediseño Orientado a Objetos de un Sistema  
Bancario Legacy en C++ hacia Java

**Autor:**

Samuel Rojas Amézquita

Código: 160005133

Correo institucional: se.rojasa@unillanos.edu.co

Asignatura: Ingeniería de Software

# Índice

|                                                                  |          |
|------------------------------------------------------------------|----------|
| <b>1. Introducción</b>                                           | <b>2</b> |
| <b>2. Descripción del sistema Legacy</b>                         | <b>2</b> |
| <b>3. Hallazgos de auditoría de memoria</b>                      | <b>2</b> |
| 3.1. Fugas de memoria por asignación dinámica . . . . .          | 2        |
| 3.2. Problemas asociados al puntero titular . . . . .            | 3        |
| <b>4. Problemas de diseño arquitectónico</b>                     | <b>3</b> |
| 4.1. Falta de encapsulamiento . . . . .                          | 3        |
| 4.2. Uso de valores numéricos para identificar cuentas . . . . . | 3        |
| 4.3. Ausencia de polimorfismo . . . . .                          | 4        |
| <b>5. Justificación del rediseño en Java</b>                     | <b>4</b> |
| 5.1. Gestión automática de memoria . . . . .                     | 4        |
| 5.2. Encapsulamiento mediante clases . . . . .                   | 4        |
| 5.3. Herencia y polimorfismo . . . . .                           | 5        |
| <b>6. Manejo de sobregiro en Cuenta Corriente</b>                | <b>5</b> |
| <b>7. Gestión de recursos mediante AutoCloseable</b>             | <b>5</b> |
| <b>8. Conclusiones</b>                                           | <b>6</b> |

# 1. Introducción

La modernización de sistemas de software antiguos representa una actividad fundamental dentro de la ingeniería de software debido a que muchos sistemas empresariales continúan funcionando sobre arquitecturas desarrolladas con lenguajes como C y C++.

El presente documento analiza un sistema bancario desarrollado originalmente en C++ con un diseño procedural basado en estructuras y manejo manual de memoria. El objetivo principal es identificar problemas relacionados con la gestión de memoria, seguridad, encapsulamiento y arquitectura, para posteriormente justificar una migración hacia un diseño orientado a objetos implementado en Java.

## 2. Descripción del sistema Legacy

El sistema inicial representa la gestión básica de cuentas bancarias mediante una estructura denominada:

```
struct CuentaLegacy
{
    char* titular;
    double saldo;
    int tipoCuenta;
    double limiteSobregiro;
};
```

La lógica del sistema se encuentra separada en funciones externas como:

- Crear cuentas mediante asignación dinámica.
- Procesar retiros modificando directamente el saldo.
- Diferenciar tipos de cuenta mediante valores numéricos.

Aunque el programa permite representar operaciones básicas, presenta problemas importantes para un sistema financiero real.

## 3. Hallazgos de auditoría de memoria

### 3.1. Fugas de memoria por asignación dinámica

Dentro de la función encargada de crear cuentas se identifican asignaciones dinámicas mediante la palabra reservada:

```
new CuentaLegacy();
new char[50];
```

Sin embargo, no existe ningún mecanismo posterior encargado de liberar estos recursos mediante instrucciones como:

```
delete  
delete []
```

Por lo tanto, la memoria reservada permanece ocupada durante la ejecución del programa, generando fugas de memoria.

### 3.2. Problemas asociados al puntero titular

El atributo:

```
char* titular;
```

representa una dirección de memoria que apunta a un arreglo dinámico de caracteres.

El problema encontrado es que dicho arreglo nunca es destruido, provocando que el espacio reservado quede perdido.

Además, el uso de:

```
strcpy(c->titular, nombre);
```

representa un riesgo debido a que no valida la longitud de la cadena ingresada.

Si el nombre supera los 50 caracteres permitidos, la función puede escribir fuera del límite del arreglo ocasionando corrupción de memoria.

## 4. Problemas de diseño arquitectónico

### 4.1. Falta de encapsulamiento

La estructura original permite modificar directamente sus atributos internos.

Por ejemplo, cualquier parte del programa podría alterar valores críticos como:

```
saldo  
limiteSobregiro  
tipoCuenta
```

Esto permite que una cuenta bancaria pueda alcanzar estados inválidos sin ningún mecanismo de validación.

### 4.2. Uso de valores numéricos para identificar cuentas

El atributo:

```
int tipoCuenta;
```

utiliza valores como:

- 1 para cuenta de ahorros.
- 2 para cuenta corriente.

Este diseño obliga al sistema a utilizar estructuras condicionales:

```
if(tipoCuenta == 1)
else if(tipoCuenta == 2)
```

El problema aparece cuando el sistema necesita agregar nuevos productos financieros, ya que sería necesario modificar constantemente la lógica existente.

### 4.3. Ausencia de polimorfismo

Cada tipo de cuenta posee reglas diferentes:

- Cuenta de ahorros: no permite saldo negativo.
- Cuenta corriente: permite sobregiro.

Sin embargo, el sistema actual concentra ambos comportamientos dentro de una misma función.

Desde una perspectiva orientada a objetos, cada tipo de cuenta debería implementar su propio comportamiento mediante herencia y polimorfismo.

## 5. Justificación del rediseño en Java

La migración hacia Java permite mejorar la seguridad y mantenibilidad del sistema mediante:

### 5.1. Gestión automática de memoria

Java utiliza un recolector de basura (Garbage Collector), eliminando la necesidad de liberar manualmente memoria reservada con instrucciones como `delete`.

Esto reduce la posibilidad de fugas de memoria generadas por errores humanos.

### 5.2. Encapsulamiento mediante clases

El nuevo diseño utiliza atributos privados y métodos controlados para modificar el estado interno de una cuenta.

De esta forma, cada objeto mantiene sus propias reglas de consistencia.

### 5.3. Herencia y polimorfismo

El rediseño propone una clase abstracta:

```
CuentaBancaria
```

con especializaciones:

```
CuentaAhorros  
CuentaCorriente
```

Cada clase implementa sus propias reglas para operaciones como:

- Retirar dinero.
- Aplicar comisiones.
- Manejar sobregiros.

## 6. Manejo de sobregiro en Cuenta Corriente

Durante la auditoría se identificó que permitir un saldo negativo no es suficiente para representar correctamente una cuenta corriente.

Un sobregiro representa una deuda temporal que debe tener seguimiento.

Por esta razón, el nuevo diseño considera que la clase `CuentaCorriente` debe ser responsable de:

- Detectar cuándo entra en sobregiro.
- Registrar el monto utilizado.
- Calcular intereses asociados.
- Mantener información necesaria para la liquidación mensual.

Esto evita que componentes externos tengan que conocer detalles internos del producto financiero.

## 7. Gestión de recursos mediante `AutoCloseable`

Aunque Java administra la memoria automáticamente, existen recursos externos como archivos de auditoría que requieren cierre explícito.

Para solucionar esto se propone implementar:

```
RegistroAuditoriaBancaria
```

utilizando la interfaz:

```
AutoCloseable
```

junto con:

```
try-with-resources
```

Esto garantiza que los recursos sean liberados correctamente sin depender del Garbage Collector.

## 8. Conclusiones

La auditoría del sistema bancario legacy permitió identificar problemas importantes relacionados con memoria, seguridad y arquitectura.

Los principales hallazgos fueron:

- Fugas de memoria por asignaciones dinámicas no liberadas.
- Riesgo de corrupción por manejo inseguro de cadenas.
- Ausencia de encapsulamiento.
- Dependencia excesiva de estructuras condicionales.
- Falta de separación de responsabilidades mediante polimorfismo.

La migración hacia Java y un diseño orientado a objetos permite construir un sistema más mantenible, seguro y preparado para futuras extensiones.

La principal mejora conceptual consiste en pasar de un modelo donde una estructura almacena datos y funciones externas modifican su estado, hacia un modelo donde cada objeto protege su información y es responsable de cumplir sus propias reglas de negocio.